PYTHON ACTIVITY BASED LEARNING

Algorithms • Flowcharts • Sample Programs • Outputs

Repository	Activity-Based-Learning-Python
Prepared for	Python practical / activity-based learning
Programs covered	9 activity examples based on the filenames visible in the repository context

Documentation note: The repository page could not be inspected directly while preparing this document. The program descriptions, code, flowcharts, and outputs below are illustrative examples inferred from the filenames, not verified copies of the repository's exact code. Run and compare each example with the original .py file before using this as a final practical record.

Contents

1. ATM Transaction Simulation (atm.py)
2. File Compression Demonstration (compression.py)
3. Duplicate Detection (duplicates.py)
4. Eligibility Checker (eligibility.py)
5. Food Ordering / Food Menu (food.py)
6. Inventory Management (inventory.py)
7. Matrix Operations (matrix.py)
8. Password Validation (password.py)
9. Pattern Printing (pattern.py)

How to use this record

Each activity includes its aim, step-by-step algorithm, a simplified flowchart, an example program, sample input, and expected output. The examples are intended to help explain the programming ideas and should be checked against the actual source files.

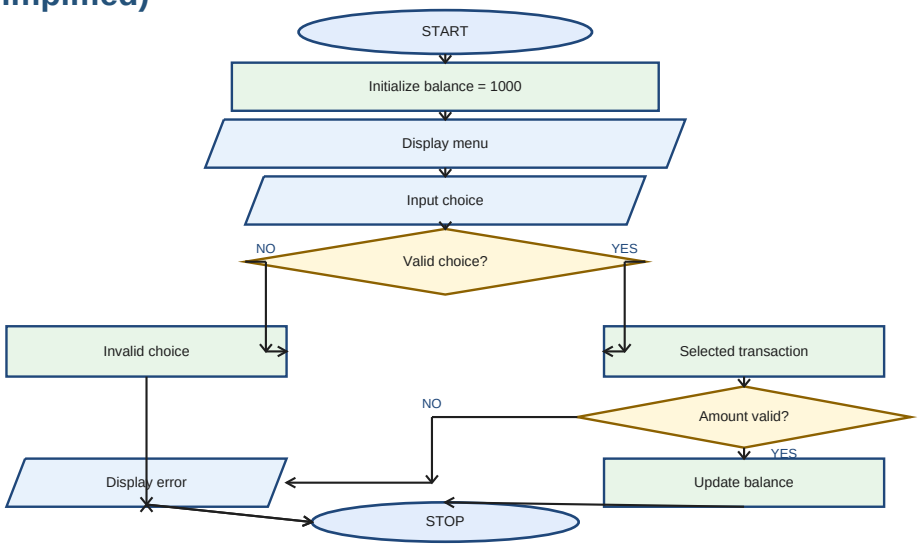
1. ATM Transaction Simulation (atm.py)

Aim
Simulate a simple ATM transaction using a menu, balance checks, and transaction choices.

Algorithm

1. Start the program.
2. Set an initial account balance.
3. Display transaction options such as balance enquiry, deposit, and withdrawal.
4. Read the user's choice and required amount.
5. Validate the amount and check available balance where relevant.
6. Update the balance and display the result.
7. Repeat for another transaction or exit.
8. Stop.

Flowchart (simplified)



Sample Python program

```
balance = 1000.0
print("ATM MENU")
print("1. Check balance")
print("2. Deposit")
print("3. Withdraw")
choice = input("Choose 1-3: ")

if choice == "1":
    print(f"Balance: Rs. {balance:.2f}")
elif choice == "2":
    amount = float(input("Deposit amount: "))
    if amount > 0:
        balance += amount
        print(f"Updated balance: Rs. {balance:.2f}")
    else:
        print("Amount must be positive")
elif choice == "3":
    amount = float(input("Withdrawal amount: "))
```

```
    if 0 < amount <= balance:
        balance -= amount
        print(f"Updated balance: Rs. {balance:.2f}")
    else:
        print("Invalid amount or insufficient balance")
else:
    print("Invalid choice")
```

Sample input

Choose 1-3: 3
Withdrawal amount: 250

Expected output

Updated balance: Rs. 750.00

Note: Illustrative example for a single transaction. The repository's exact source file was not available to inspect in this session; this example documents the activity suggested by the filename.

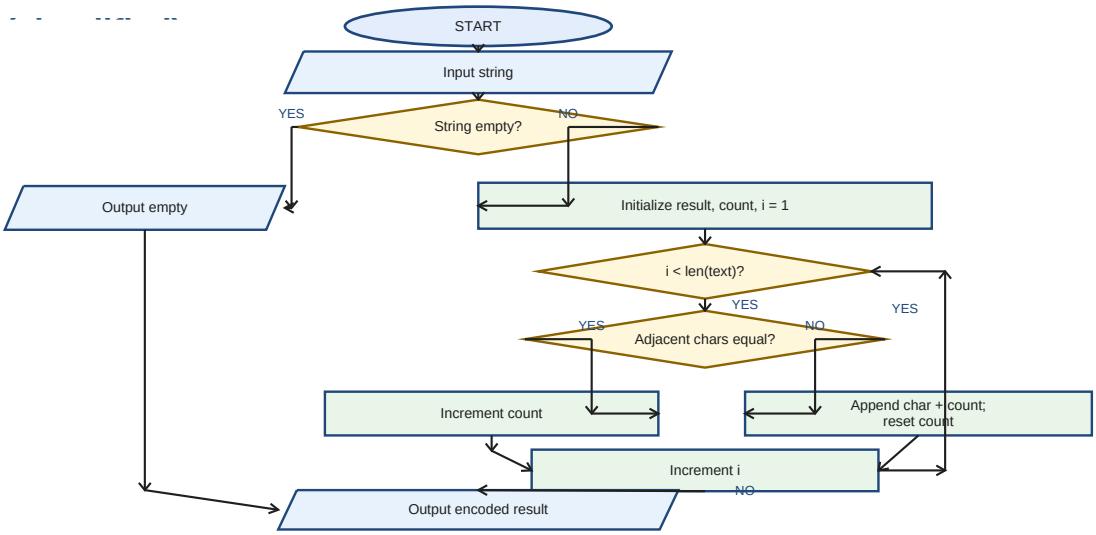
2. File Compression Demonstration (compression.py)

Aim
Demonstrate a basic lossless compression idea by encoding repeated adjacent characters as character-count pairs (run-length encoding).

Algorithm

1. Start.
2. Read a string.
3. Initialize an empty encoded result.
4. Scan the string from left to right, counting adjacent repeated characters.
5. Append each character and its count to the result.
6. Display the encoded result.
7. Stop.

Flowcha



Sample Python program

```
def run_length_encode(text):
    if not text:
        return ""
    result = []
    count = 1
    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            result.append(text[i - 1] + str(count))
            count = 1
    result.append(text[-1] + str(count))
    return "".join(result)

sample = "aaabbc"
print("Original:", sample)
print("Encoded:", run_length_encode(sample))
```

Sample input

Sample string: aaabbc

Expected output

Original: aaabbc
Encoded: a3b2c1

Note: This is a teaching example of run-length encoding, not a claim about the exact algorithm in the repository's file.

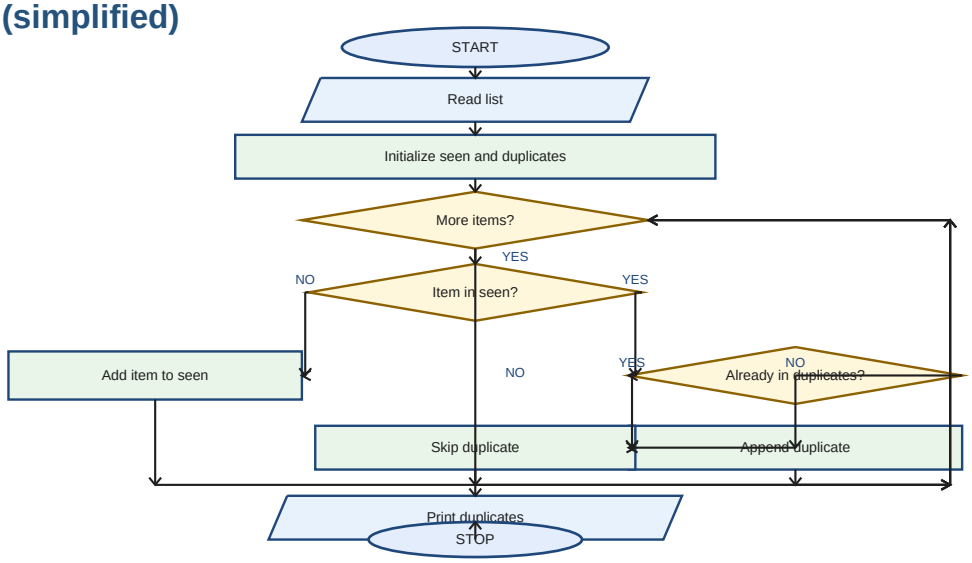
3. Duplicate Detection (duplicates.py)

Aim
Identify repeated values in a sequence.

Algorithm

1. Start.
2. Create or read a list of values.
3. Create an empty set for seen values and another collection for duplicates.
4. Visit each value in order.
5. If it has already been seen, record it as a duplicate; otherwise add it to seen.
6. Display duplicate values.
7. Stop.

Flowchart (simplified)



Sample Python program

```
values = [4, 2, 7, 2, 4, 9, 2]
seen = set()
duplicates = []

for value in values:
    if value in seen:
        if value not in duplicates:
            duplicates.append(value)
    else:
        seen.add(value)

print("Values:", values)
print("Duplicates:", duplicates)
```

Sample input

[4, 2, 7, 2, 4, 9, 2]

Expected output

Values: [4, 2, 7, 2, 4, 9, 2]
Duplicates: [2, 4]

Note: The example reports each duplicated value once, in the order it is first detected as a repeat.

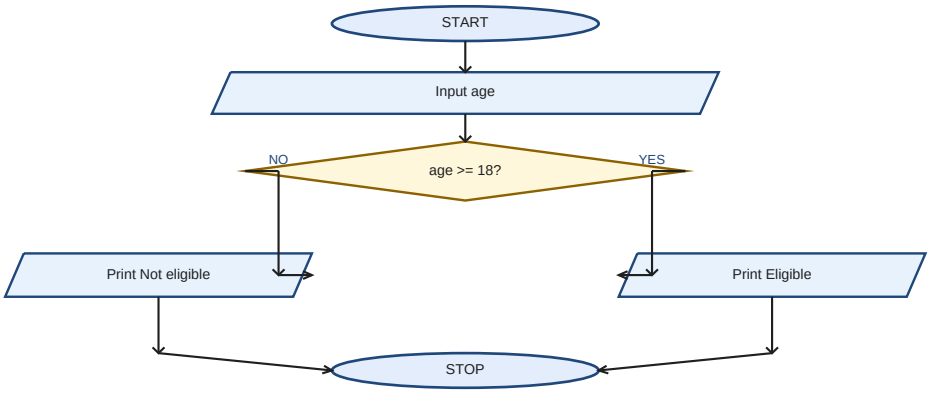
4. Eligibility Checker (eligibility.py)

Aim
Use conditional statements to decide whether a person meets a stated eligibility threshold.

Algorithm

1. Start.
2. Read the required input, such as age.
3. Check the input against the eligibility condition.
4. Display eligible or not eligible.
5. Stop.

Flowchart (simplified)



Sample

```
age = int(input("Enter age: "))
if age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

Sample input

Enter age: 20

Expected output

Eligible

Note: The threshold is illustrative. Confirm the intended rule against the actual repository code before submission.

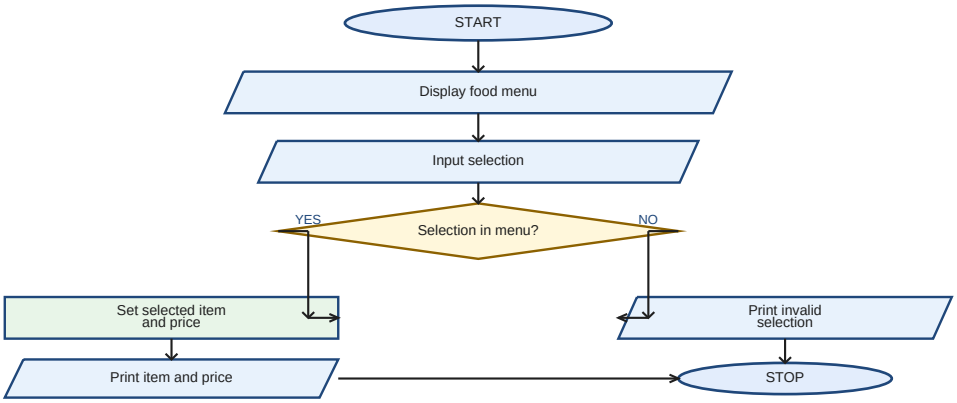
5. Food Ordering / Food Menu (food.py)

Aim
Display a food menu, accept a selection, and calculate the price of the chosen item.

Algorithm

1. Start.
2. Display available food items and prices.
3. Read the selected item number.
4. Check whether the selection is valid.
5. If valid, display the selected item and price; otherwise show an error.
6. Stop.

Flowchart (simplified)



Sample Python program

```
menu = {
    1: ("Idli", 30),
    2: ("Dosa", 50),
    3: ("Meals", 90)
}
print("1. Idli - Rs.30")
print("2. Dosa - Rs.50")
print("3. Meals - Rs.90")
choice = int(input("Select item: "))

if choice in menu:
    item, price = menu[choice]
    print(f"You selected {item}. Total: Rs.{price}")
else:
    print("Invalid selection")
```

Sample input

Select item: 2

Expected output

You selected Dosa. Total: Rs.50

Note: Example menu and prices are illustrative.

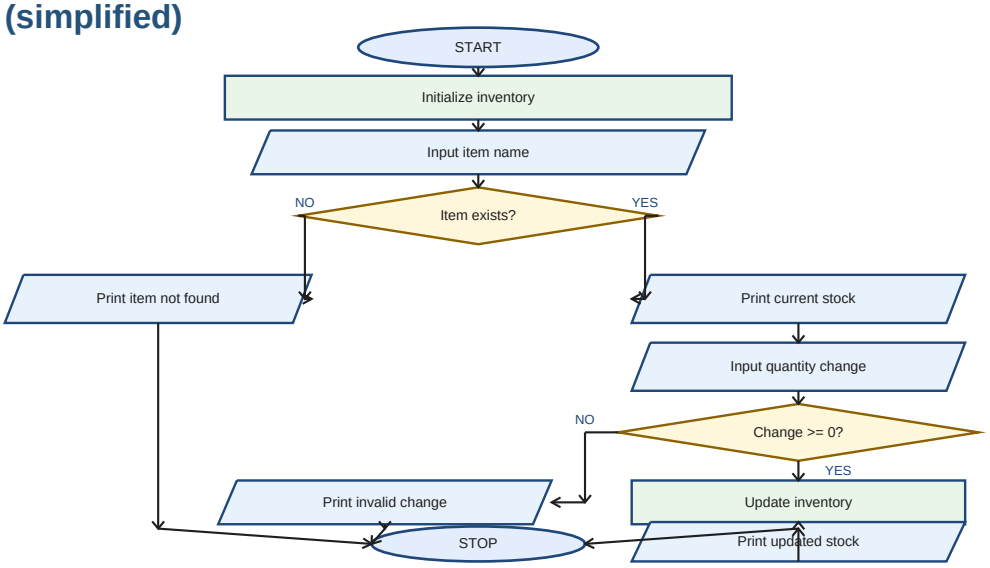
6. Inventory Management (inventory.py)

Aim
Maintain item quantities and support a simple stock lookup or update.

Algorithm

1. Start.
2. Initialize an inventory dictionary with item names and quantities.
3. Display or request an item name.
4. Check whether the item exists.
5. Read a quantity change if required and ensure the new quantity is not negative.
6. Update and display inventory.
7. Stop.

Flowchart (simplified)



Sample Python program

```
inventory = {"pen": 10, "notebook": 5, "pencil": 20}
item = input("Enter item: ").lower()

if item in inventory:
    print("Current stock:", inventory[item])
    change = int(input("Quantity to add: "))
    if change >= 0:
        inventory[item] += change
        print("Updated stock:", inventory[item])
    else:
        print("Quantity change must be non-negative")
else:
    print("Item not found")
```

Sample input

Enter item: pen
Quantity to add: 5

Expected output

Current stock: 10
Updated stock: 15

Note: This sample demonstrates stock lookup and adding quantity; it is not a verified reproduction of the repository's exact implementation.

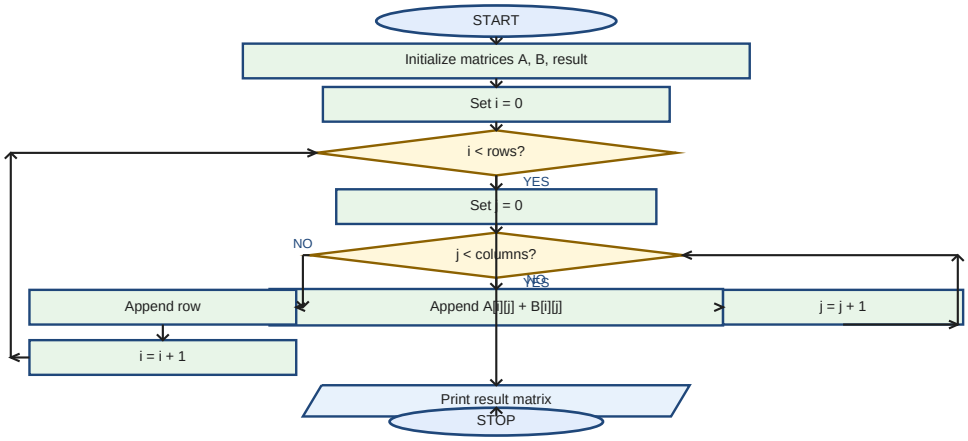
7. Matrix Operations (matrix.py)

Aim
Represent a matrix using nested lists and perform a basic operation such as matrix addition.

Algorithm

1. Start.
2. Define or read two matrices of equal dimensions.
3. Create an empty result matrix.
4. For each row and column, add corresponding elements.
5. Display the result matrix.
6. Stop.

Flowchart (simplified)



Sample Python program

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
result = []

for i in range(len(A)):
    row = []
    for j in range(len(A[0])):
        row.append(A[i][j] + B[i][j])
    result.append(row)

print("Matrix sum:")
for row in result:
    print(row)
```

Sample input

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

Expected output

Matrix sum:
[6, 8]
[10, 12]

Note: Matrix addition is an illustrative activity; verify whether the source file performs addition or another matrix operation.

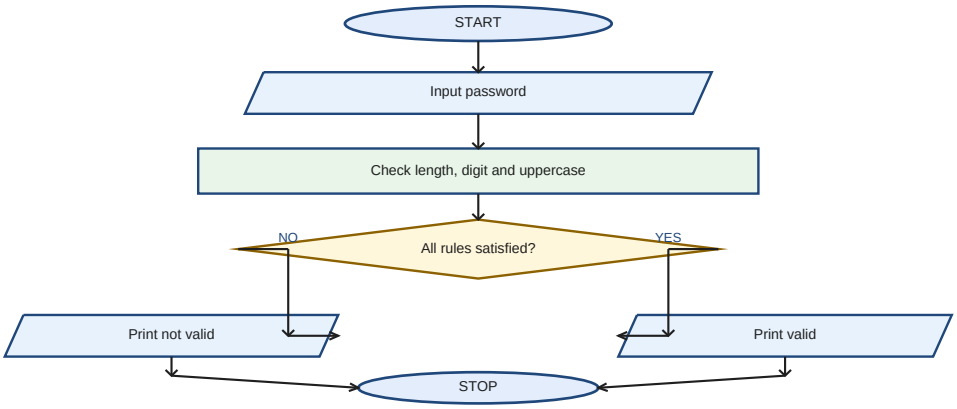
8. Password Validation (password.py)

Aim
Check whether a password meets a simple set of strength rules.

Algorithm

1. Start.
2. Read a password.
3. Check minimum length and the required character categories.
4. If all required checks pass, report valid/strong; otherwise report which requirements are missing.
5. Stop.

Flowchart (simplified)



Sample Python program

```
password = input("Enter password: ")
has_length = len(password) >= 8
has_digit = any(ch.isdigit() for ch in password)
has_upper = any(ch.isupper() for ch in password)

if has_length and has_digit and has_upper:
    print("Password meets the sample rules")
else:
    print("Password does not meet the sample rules")
```

Sample input

Enter password: LearnPython7

Expected output

Password meets the sample rules

Note: These are sample validation rules for demonstration only; use the source file's actual criteria if they differ.

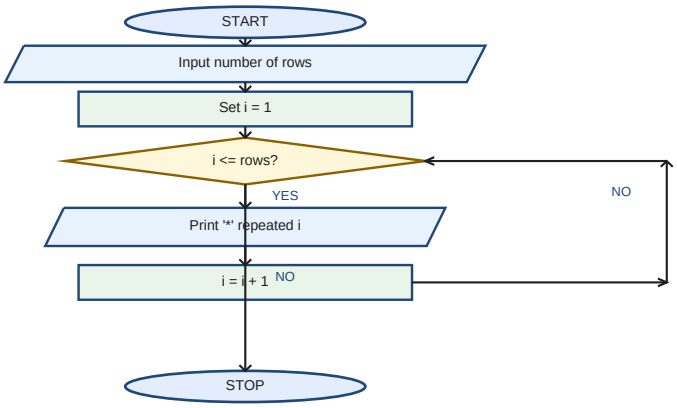
9. Pattern Printing (pattern.py)

Aim
Use nested loops to print a simple star pattern.

Algorithm

1. Start.
2. Read the number of rows.
3. For each row from 1 through the requested row count, print that many stars.
4. Move to the next line after each row.
5. Stop.

Flowchart (simplified)



Sample Python program

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i)
```

Sample input

Enter number of rows: 4

Expected output

```
*
**
***
****
```

Note: A simple increasing triangle is shown; confirm the exact pattern expected by the repository's source file.

Conclusion

These activities illustrate fundamental Python skills: conditional logic, loops, lists and sets, dictionaries, string processing, input/output, and matrix representation. Algorithms help organize the solution before coding, flowcharts show the sequence of actions, and sample outputs make it easier to verify behavior.

Source repository

https://github.com/saikarthik-1736/Activity-Based-Learning-Python

For an exact repository-matched edition, provide the .py source files or upload a ZIP of the repository. Then the examples, algorithms, outputs, and flowcharts can be aligned with each original program.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

1. ATM Transaction Simulation

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
SET balance = 1000
DISPLAY ATM menu
INPUT choice

IF choice = "1" THEN
    DISPLAY balance
ELSE IF choice = "2" THEN
    INPUT deposit amount
    IF amount > 0 THEN
        balance = balance + amount
        DISPLAY updated balance
    ELSE
        DISPLAY "Amount must be positive"
    END IF
ELSE IF choice = "3" THEN
    INPUT withdrawal amount
    IF amount > 0 AND amount <= balance THEN
        balance = balance - amount
        DISPLAY updated balance
    ELSE
        DISPLAY "Invalid amount or insufficient balance"
    END IF
ELSE
    DISPLAY "Invalid choice"
END IF
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

2. File Compression Demonstration

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
DEFINE RUN_LENGTH_ENCODE(text)
IF text is empty THEN
    RETURN empty string
END IF
SET result = empty list
SET count = 1

FOR each index i from 1 to length(text) - 1
    IF text[i] = text[i - 1] THEN
        INCREMENT count
    ELSE
        APPEND text[i - 1] and count to result
        SET count = 1
    END IF
END FOR

APPEND last character and count to result
RETURN joined result
END DEFINE

SET sample = "aaabbc"
DISPLAY original string
DISPLAY encoded string
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

3. Duplicate Detection

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
SET values = [4, 2, 7, 2, 4, 9, 2]
SET seen = empty set
SET duplicates = empty list

FOR each value in values
    IF value is in seen THEN
        IF value is not in duplicates THEN
            APPEND value to duplicates
        END IF
    ELSE
        ADD value to seen
    END IF
END FOR

DISPLAY values
DISPLAY duplicates
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

4. Eligibility Checker

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
INPUT age
IF age >= 18 THEN
    DISPLAY "Eligible"
ELSE
    DISPLAY "Not eligible"
END IF
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

5. Food Ordering / Food Menu

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
CREATE menu with item numbers, names and prices
DISPLAY food menu
INPUT choice

IF choice is present in menu THEN
    GET item name and price from menu
    DISPLAY selected item and total price
ELSE
    DISPLAY "Invalid selection"
END IF
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

6. Inventory Management

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
INITIALIZE inventory with item names and quantities
INPUT item name
CONVERT item name to lowercase

IF item exists in inventory THEN
    DISPLAY current stock
    INPUT quantity change
    IF quantity change >= 0 THEN
        UPDATE stock = current stock + quantity change
        DISPLAY updated stock
    ELSE
        DISPLAY "Quantity change must be non-negative"
    END IF
ELSE
    DISPLAY "Item not found"
END IF
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

7. Matrix Operations

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
DEFINE matrix A and matrix B
INITIALIZE result as empty list

FOR each row index i in matrix A
    INITIALIZE row as empty list
    FOR each column index j in the row
        CALCULATE sum = A[i][j] + B[i][j]
        APPEND sum to row
    END FOR
    APPEND row to result
END FOR

DISPLAY "Matrix sum"
FOR each row in result
    DISPLAY row
END FOR
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

8. Password Validation

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
INPUT password
SET has_length = length(password) >= 8
SET has_digit = password contains a digit
SET has_upper = password contains an uppercase letter

IF has_length AND has_digit AND has_upper THEN
    DISPLAY "Password meets the sample rules"
ELSE
    DISPLAY "Password does not meet the sample rules"
END IF
STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Pseudocode

9. Pattern Printing

The following pseudocode follows the sample Python program and algorithm in the practical record.

```
START
INPUT number of rows

FOR i from 1 to number of rows
    CREATE a line containing i asterisks
    DISPLAY the line
END FOR

STOP
```

Note: The PDF describes these programs as illustrative examples. Verify the logic against the original repository source before submission.

PYTHON ACTIVITY BASED LEARNING

Conclusion

The nine Python activities demonstrate important programming concepts, including input and output operations, conditional statements, loops, strings, lists, sets, dictionaries, and matrix operations. Each activity follows a problem-solving sequence: understand the problem, prepare an algorithm, represent the logic with a flowchart, write the program, and check the output.

Writing pseudocode before coding helps express the program logic in a simple, language-independent form. Flowcharts provide a visual representation of the same logic, while Python programs implement it. Together, algorithms, pseudocode, flowcharts, programs, and sample outputs make the activities easier to understand, test, and maintain.

Through these activities, the learner gains practice in breaking a problem into smaller steps, selecting suitable data structures, applying decisions and repetition, and checking whether a program produces the expected result. The concepts form a foundation for solving more complex problems using Python.

End of Practical Record